Exercise: Building a Fully Connected Network (FCN) for Devnagari Digit Classification.

Task 1: Data Preparation

Loading the Data

```
import os
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.utils import to_categorical
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image

print(f"TensorFlow version : {tf.__version__}")
print(f"Keras version      : {tf.keras.__version__}")
print(f"GPU Available      : {tf.config.list_physical_devices('GPU')}")
```

```
TensorFlow version : 2.19.0
Keras version      : 3.13.2
GPU Available      : []
```

```
#  Dataset paths
DATASET_ROOT = '/content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDataset'
train_dir    = os.path.join(DATASET_ROOT, 'Train')
test_dir     = os.path.join(DATASET_ROOT, 'Test')

# Enhanced verification for debugging
if not os.path.exists(DATASET_ROOT):
    print(f"Error: The specified DATASET_ROOT folder does not exist: {DATASET_ROOT}")
    print("Please ensure your Google Drive is mounted and the path is correct.")
    print("You can mount your Drive using: 'from google.colab import drive; drive.mount('/content/drive')' in a previous cell
    raise FileNotFoundError(f"DATASET_ROOT not found: {DATASET_ROOT}")
else:
    print(f"DATASET_ROOT found: {DATASET_ROOT}")
    print(f"Contents of DATASET_ROOT: {os.listdir(DATASET_ROOT)}")

# Verify both train and test directories exist
if not os.path.exists(train_dir):
    raise AssertionError(f"Train folder not found: {train_dir}. Please check the folder name 'Train' within '{DATASET_ROOT}'.

if not os.path.exists(test_dir):
    raise AssertionError(f"Test folder not found: {test_dir}. Please check the folder name 'Test' within '{DATASET_ROOT}'.")

print(f"Dataset root  : {DATASET_ROOT}")
print(f"Train folder  : {train_dir}")
print(f"Test folder   : {test_dir}")

classes = sorted(os.listdir(train_dir))
print(f"Classes found : {len(classes)}  ->  {classes}")
```

```
DATASET_ROOT found: /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDataset
Contents of DATASET_ROOT: ['Test', 'Train', 'best_devnagari_model.h5', 'devnagari_fcn_model.h5']
Dataset root  : /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDataset
Train folder  : /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDataset/Train
Test folder   : /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDataset/Test
Classes found : 10  ->  ['digit_0', 'digit_1', 'digit_2', 'digit_3', 'digit_4', 'digit_5', 'digit_6', 'digit_7', 'digit_8', 'd
```

Step 1b: Load and Preprocess Images Using PIL

```
def load_images_from_folder(folder, img_size=(32, 32)):
    images = []
    labels = []
    for label_idx, class_name in enumerate(classes):
        print(f"  Processing class: {class_name} (label {label_idx}) in {folder}...")
        class_folder = os.path.join(folder, class_name)
```

```
        if not os.path.exists(class_folder):
            print(f"  Warning: Class folder not found: {class_folder}. Skipping.")
            continue

        class_images_count = 0
        for filename in os.listdir(class_folder):
            img_path = os.path.join(class_folder, filename)
            try:
                img = Image.open(img_path).convert('L')  # Convert to grayscale
                img = img.resize(img_size)
                images.append(np.array(img))
                labels.append(label_idx)
                class_images_count += 1
            except Exception as e:
                print(f"Error loading image {img_path}: {e}")
        print(f"  Loaded {class_images_count} images for class {class_name}.")
    return np.array(images), np.array(labels)

# Load training and testing data
print("Loading training images...")
X_train, y_train = load_images_from_folder(train_dir)
print("Finished loading training images.") # Added to confirm completion
print("Loading testing images...")
X_test, y_test = load_images_from_folder(test_dir)
print("Finished loading testing images.") # Added to confirm completion

print(f"Original X_train shape: {X_train.shape}")
print(f"Original y_train shape: {y_train.shape}")
print(f"Original X_test shape: {X_test.shape}")
print(f"Original y_test shape: {y_test.shape}")

# Normalize images to [0, 1] range
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0

# Reshape images to (num_samples, height, width, channels) for Keras
X_train = np.expand_dims(X_train, axis=-1)
X_test = np.expand_dims(X_test, axis=-1)

# One-hot encode the labels
y_train = to_categorical(y_train, num_classes=len(classes))
y_test = to_categorical(y_test, num_classes=len(classes))

print(f"Processed X_train shape: {X_train.shape}")
print(f"Processed y_train shape: {y_train.shape}")
print(f"Processed X_test shape: {X_test.shape}")
print(f"Processed y_test shape: {y_test.shape}")
```

```
Loading training images...
  Processing class: digit_0 (label 0) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 1700 images for class digit_0.
  Processing class: digit_1 (label 1) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 1700 images for class digit_1.
  Processing class: digit_2 (label 2) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 1700 images for class digit_2.
  Processing class: digit_3 (label 3) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 1700 images for class digit_3.
  Processing class: digit_4 (label 4) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 1700 images for class digit_4.
  Processing class: digit_5 (label 5) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 1700 images for class digit_5.
  Processing class: digit_6 (label 6) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 1700 images for class digit_6.
  Processing class: digit_7 (label 7) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 1700 images for class digit_7.
  Processing class: digit_8 (label 8) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 1700 images for class digit_8.
  Processing class: digit_9 (label 9) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 1700 images for class digit_9.
Finished loading training images.
Loading testing images...
  Processing class: digit_0 (label 0) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 300 images for class digit_0.
  Processing class: digit_1 (label 1) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 300 images for class digit_1.
  Processing class: digit_2 (label 2) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 300 images for class digit_2.
  Processing class: digit_3 (label 3) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 300 images for class digit_3.
```

```
  Processing class: digit_4 (label 4) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 300 images for class digit_4.
  Processing class: digit_5 (label 5) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 300 images for class digit_5.
  Processing class: digit_6 (label 6) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 300 images for class digit_6.
  Processing class: digit_7 (label 7) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 300 images for class digit_7.
  Processing class: digit_8 (label 8) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 300 images for class digit_8.
  Processing class: digit_9 (label 9) in /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDatase
  Loaded 300 images for class digit_9.
Finished loading testing images.
Original X_train shape: (17000, 32, 32)
Original y_train shape: (17000,)
Original X_test shape: (3000, 32, 32)
Original y_test shape: (3000,)
Processed X_train shape: (17000, 32, 32, 1)
Processed y_train shape: (17000, 10)
Processed X_test shape: (3000, 32, 32, 1)
Processed y_test shape: (3000, 10)
```

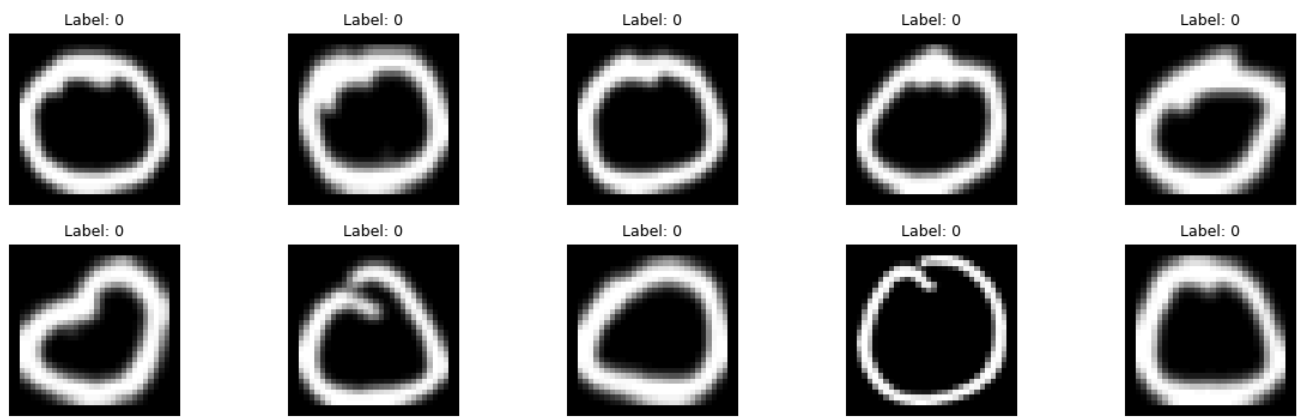
Step 1c: Reshape (Flatten) and One-Hot Encode

```
IMG_HEIGHT = X_train.shape[1]
IMG_WIDTH = X_train.shape[2]
INPUT_DIM = IMG_HEIGHT * IMG_WIDTH
NUM_CLASSES = len(classes)

# Flatten images into vectors for FCN input
x_train = X_train.reshape(-1, INPUT_DIM)
x_test  = X_test.reshape(-1, INPUT_DIM)

# y_train and y_test are already one-hot encoded from the previous step.

print(f"x_train : {x_train.shape}   (flattened + normalised)")
print(f"y_train : {y_train.shape}   (one-hot encoded)")
print(f"x_test  : {x_test.shape}")
print(f"y_test  : {y_test.shape}")
print(f"Pixel range : [{x_train.min():.2f}, {x_train.max():.2f}]")
```

```
x_train : (17000, 1024)   (flattened + normalised)
y_train : (17000, 10)   (one-hot encoded)
x_test  : (3000, 1024)
y_test  : (3000, 10)
Pixel range : [0.00, 1.00]
```

Step 1d: Visualise Sample Training Images

```
plt.figure(figsize=(12, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_train[i].squeeze(), cmap='gray') # Use X_train for original image and squeeze to remove channel dim
    plt.title(f"Label: {np.argmax(y_train[i])}", fontsize=9) # Convert one-hot to integer label
    plt.axis('off')
plt.suptitle('Sample Training Images — Devnagari Digits', fontsize=13, y=1.02)
plt.tight_layout()
plt.show()
```



Task 2: Build the FCN Model

```
model = keras.Sequential(
    [
        keras.layers.Input(shape=(INPUT_DIM,)),

        # Hidden Layer 1: 64 neurons, sigmoid
        keras.layers.Dense(64,  activation='sigmoid', name='hidden_1'),

        # Hidden Layer 2: 128 neurons, sigmoid
        keras.layers.Dense(128, activation='sigmoid', name='hidden_2'),

        # Hidden Layer 3: 256 neurons, sigmoid
        keras.layers.Dense(256, activation='sigmoid', name='hidden_3'),

        # Output: 10 classes, softmax converts logits to probabilities
        keras.layers.Dense(NUM_CLASSES, activation='softmax', name='output'),
    ],
    name='Devnagari_FCN'
)

model.summary()
```

Model: "Devnagari_FCN"

Layer (type)	Output Shape	Param #
hidden_1 (Dense)	(None, 64)	65,600
hidden_2 (Dense)	(None, 128)	8,320
hidden_3 (Dense)	(None, 256)	33,024
output (Dense)	(None, 10)	2,570

Total params: 109,514 (427.79 KB)
Trainable params: 109,514 (427.79 KB)

Task 3: Compile the Model

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

print("Model compiled:")
print("  Optimizer : Adam")
print("  Loss      : Categorical Cross-Entropy")
print("  Metric    : Accuracy")
```

```
Model compiled:
  Optimizer : Adam
```

```
  Loss      : Categorical Cross-Entropy
  Metric    : Accuracy
```

Task 4: Train the Model

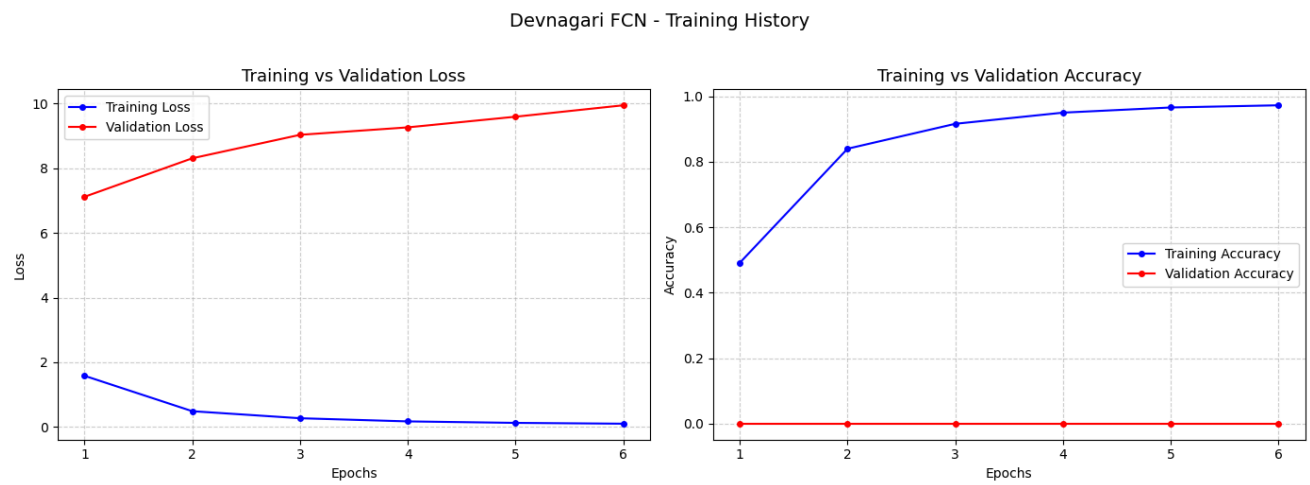
```
BATCH_SIZE = 128
EPOCHS     = 20

# Best-model checkpoint saved directly to your Google Drive folder
CHECKPOINT_PATH = os.path.join(DATASET_ROOT, 'best_devnagari_model.h5')

callbacks = [
    keras.callbacks.ModelCheckpoint(
        filepath=CHECKPOINT_PATH,
        monitor='val_accuracy',
        save_best_only=True,
        verbose=1
    ),
    keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=5,
        restore_best_weights=True,
        verbose=1
    ),
]

print(f"Training: batch_size={BATCH_SIZE}, epochs={EPOCHS}, validation_split=0.2")
print("-" * 60)

history = model.fit(
    x_train,
    y_train,
    batch_size=BATCH_SIZE,
    epochs=EPOCHS,
    validation_split=0.2,
    callbacks=callbacks,
    verbose=1
)
```

```
Training: batch_size=128, epochs=20, validation_split=0.2
------------------------------------------------------------
Epoch 1/20
106/107 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.2984 - loss: 1.9479
Epoch 1: val_accuracy improved from None to 0.00000, saving model to /content/drive/MyDrive/AI L6/Copy of devnagari digit/Deva
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file forma

Epoch 1: finished saving model to /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDataset/best_
107/107 ━━━━━━━━━━━━━━━━━━━━ 3s 15ms/step - accuracy: 0.4913 - loss: 1.5818 - val_accuracy: 0.0000e+00 - val_loss: 7.1180
Epoch 2/20
103/107 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8147 - loss: 0.6069
Epoch 2: val_accuracy did not improve from 0.00000
107/107 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 0.8397 - loss: 0.4878 - val_accuracy: 0.0000e+00 - val_loss: 8.3080
Epoch 3/20
100/107 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8978 - loss: 0.3051
Epoch 3: val_accuracy did not improve from 0.00000
107/107 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9158 - loss: 0.2702 - val_accuracy: 0.0000e+00 - val_loss: 9.0329
Epoch 4/20
104/107 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9456 - loss: 0.1901
Epoch 4: val_accuracy did not improve from 0.00000
107/107 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9499 - loss: 0.1730 - val_accuracy: 0.0000e+00 - val_loss: 9.2634
Epoch 5/20
107/107 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9645 - loss: 0.1300
Epoch 5: val_accuracy did not improve from 0.00000
107/107 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9657 - loss: 0.1271 - val_accuracy: 0.0000e+00 - val_loss: 9.5927
Epoch 6/20
 98/107 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9700 - loss: 0.1123
Epoch 6: val_accuracy did not improve from 0.00000
107/107 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9724 - loss: 0.1017 - val_accuracy: 0.0000e+00 - val_loss: 9.9456
Epoch 6: early stopping
Restoring model weights from the end of the best epoch: 1.
```

Visualise Training History

```
train_loss  = history.history['loss']
val_loss    = history.history['val_loss']
train_acc   = history.history['accuracy']
```

```
val_acc     = history.history['val_accuracy']
epoch_range = range(1, len(train_loss) + 1)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# --- Loss ---
axes[0].plot(epoch_range, train_loss, 'b-o', markersize=4, label='Training Loss')
axes[0].plot(epoch_range, val_loss,   'r-o', markersize=4, label='Validation Loss')
axes[0].set_title('Training vs Validation Loss', fontsize=13)
axes[0].set_xlabel('Epochs')
axes[0].set_ylabel('Loss')
axes[0].legend()
axes[0].grid(True, linestyle='--', alpha=0.6)

# --- Accuracy ---
axes[1].plot(epoch_range, train_acc, 'b-o', markersize=4, label='Training Accuracy')
axes[1].plot(epoch_range, val_acc,   'r-o', markersize=4, label='Validation Accuracy')
axes[1].set_title('Training vs Validation Accuracy', fontsize=13)
axes[1].set_xlabel('Epochs')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].grid(True, linestyle='--', alpha=0.6)

plt.suptitle('Devnagari FCN - Training History', fontsize=14, y=1.02)
plt.tight_layout()
plt.savefig('training_history.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: training_history.png")
```




```
Saved: training_history.png
```

Task 5: Evaluate the Model on Test Data

```
print("Evaluating on test set...")
print("-" * 40)
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print("-" * 40)
print(f"Test Loss     : {test_loss:.4f}")
print(f"Test Accuracy : {test_acc * 100:.2f}%")
```

```
Evaluating on test set...
----------------------------------------
94/94 - 0s - 2ms/step - accuracy: 0.6527 - loss: 2.0399
----------------------------------------
Test Loss     : 2.0399
Test Accuracy : 65.27%
```

Task 6: Save and Load the Model

```
MODEL_PATH = os.path.join(DATASET_ROOT, 'devnagari_fcn_model.h5')

model.save(MODEL_PATH)
print(f"Model saved to: {MODEL_PATH}")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file forma
Model saved to: /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDataset/devnagari_fcn_model.h5
```

```
# Load the saved model back and re-evaluate
loaded_model = tf.keras.models.load_model(MODEL_PATH)
print(f"Model loaded from: {MODEL_PATH}")

print("\nRe-evaluating loaded model...")
loaded_loss, loaded_acc = loaded_model.evaluate(x_test, y_test, verbose=2)
print(f"\nLoaded Model - Test Loss     : {loaded_loss:.4f}")
print(f"Loaded Model - Test Accuracy : {loaded_acc * 100:.2f}%")
```

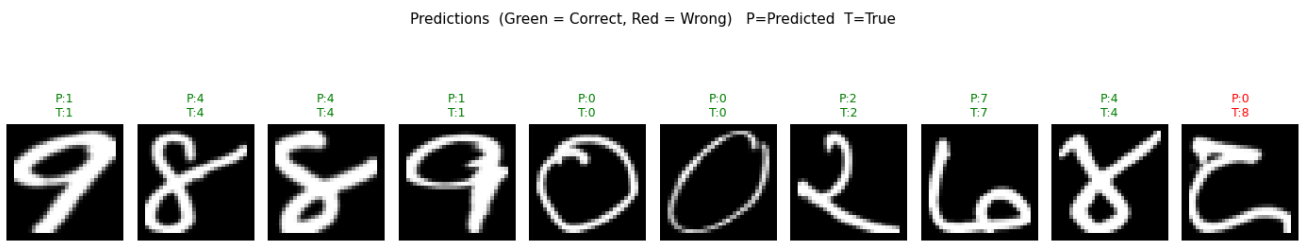
```
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty u
Model loaded from: /content/drive/MyDrive/AI L6/Copy of devnagari digit/DevanagariHandwrittenDigitDataset/devnagari_fcn_model.

Re-evaluating loaded model...
94/94 - 1s - 6ms/step - accuracy: 0.6527 - loss: 2.0399

Loaded Model - Test Loss     : 2.0399
Loaded Model - Test Accuracy : 65.27%
```

Task 7: Make Predictions

```
# Generate predictions on the full test set
predictions      = loaded_model.predict(x_test, verbose=0)
predicted_labels = np.argmax(predictions, axis=1)
true_labels      = np.argmax(y_test,      axis=1)

print(f"Predictions shape  : {predictions.shape}")
print(f"First 10 predicted : {predicted_labels[:10]}")
print(f"First 10 true      : {true_labels[:10]}")
```

```
Predictions shape  : (3000, 10)
First 10 predicted : [0 0 0 0 0 0 0 0 0 0]
First 10 true      : [0 0 0 0 0 0 0 0 0 0]
```

```
# Visualise 10 random test predictions
num_samples = 10
indices = np.random.choice(len(x_test), num_samples, replace=False)

plt.figure(figsize=(14, 3))
for plot_idx, idx in enumerate(indices):
    img        = x_test[idx].reshape(IMG_HEIGHT, IMG_WIDTH)
    pred_label = predicted_labels[idx]
    true_label = true_labels[idx]
    color      = 'green' if pred_label == true_label else 'red'

    plt.subplot(1, num_samples, plot_idx + 1)
    plt.imshow(img, cmap='gray')
    plt.title(f"P:{pred_label}\nT:{true_label}", color=color, fontsize=9)
    plt.axis('off')

plt.suptitle('Predictions  (Green = Correct, Red = Wrong)   P=Predicted  T=True',
             fontsize=11, y=1.05)
plt.tight_layout()
plt.savefig('predictions_sample.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: predictions_sample.png")
```

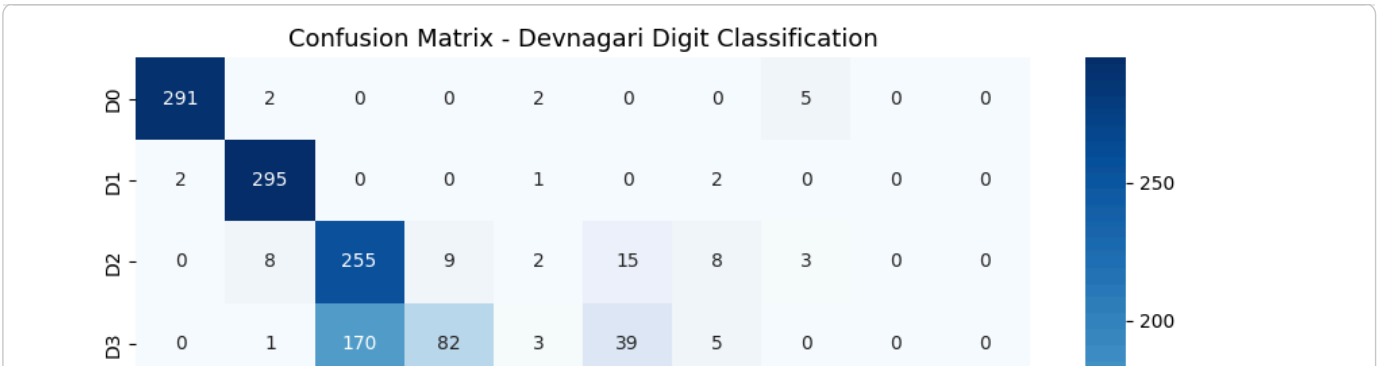



```
Saved: predictions_sample.png
```

```python
# Per-class classification report
print("Classification Report:")
print("-" * 60)
print(classification_report(
    true_labels,
    predicted_labels,
    target_names=[f"Digit {i}" for i in range(NUM_CLASSES)]
))
```

```
Classification Report:
------------------------------------------------------------
              precision    recall  f1-score   support

     Digit 0       0.61      0.97      0.75       300
     Digit 1       0.53      0.98      0.69       300
     Digit 2       0.52      0.85      0.64       300
     Digit 3       0.88      0.27      0.42       300
     Digit 4       0.68      0.97      0.80       300
     Digit 5       0.80      0.85      0.82       300
     Digit 6       0.77      0.82      0.79       300
     Digit 7       0.78      0.81      0.79       300
     Digit 8       0.00      0.00      0.00       300
     Digit 9       0.00      0.00      0.00       300

    accuracy                           0.65      3000
   macro avg       0.56      0.65      0.57      3000
weighted avg       0.56      0.65      0.57      3000

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defi
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defi
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defi
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```python
# Confusion matrix heatmap
cm = confusion_matrix(true_labels, predicted_labels)

plt.figure(figsize=(9, 7))
sns.heatmap(
    cm, annot=True, fmt='d', cmap='Blues',
    xticklabels=[f"D{i}" for i in range(NUM_CLASSES)],
    yticklabels=[f"D{i}" for i in range(NUM_CLASSES)]
)
plt.title('Confusion Matrix - Devnagari Digit Classification', fontsize=13)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: confusion_matrix.png")
```


Confusion Matrix - Devnagari Digit Classification
D0
D1
D2
D3
291 2 0 0 2 0 0 5 0 0
2 295 0 0 1 0 2 0 0 0
0 8 255 9 2 15 8 3 0 0
0 1 170 82 3 39 5 0 0 0
250
200